

Билет 11. `select`, `poll`, `epoll`. Разница между ними. Многопоточный сервер через `epoll`

0. Формулировка билета

Системные вызовы `select`, `poll` и `epoll`. Разница между ними. Реализация многопоточного сервера через `epoll`.

Главная идея:

В busy loop polling программа сама постоянно обходит все fd и спрашивает: “готов ли ты?”

`select`, `poll` и `epoll` позволяют спросить ядро: “усыпи меня, пока хотя бы один fd не станет готов, а потом верни список готовых”.

Ключевая схема:

```
busy loop:
  for every fd:
    read/write
    EAGAIN
  next fd

I/O multiplexing:
  wait for ready fds
  process only ready fds
```

1. Зачем нужны `select`, `poll`, `epoll`

В прошлом билете мы перевели сокеты в неблокирующий режим.

Теперь `read` работает так:

```
данные есть      → read возвращает байты
данных нет       → read возвращает -1, errno = EAGAIN
соединение закрыто → read возвращает 0
```

Это решило проблему блокировки worker-а на одном молчащем клиенте.

Но появилась новая проблема: busy loop.

```
while (true) {  
    for (auto& client : clients) {  
        read(client.fd, ...);  
        write(client.fd, ...);  
    }  
}
```

Если клиентов 10 000, а активен один, мы всё равно проверяем все 10 000.

Проблемы:

CPU burn
O(n) обход всех клиентов
много лишних syscalls с EAGAIN

Хотелось бы:

спать, пока ничего не происходит
проснуться, когда хотя бы один fd готов
получить список готовых fd
обрабатывать только их

Это и есть I/O multiplexing.

2. Что значит “fd готов”

Важно: `select` / `poll` / `epoll` обычно сообщают не “операция уже выполнена”, а **готовность**.

Это reactor-модель:

ядро говорит: fd готов к чтению
пользовательский код сам вызывает read

ядро говорит: fd готов к записи
пользовательский код сам вызывает write

Пример:

EPOLLIN:
можно попробовать читать
read, скорее всего, не заблокируется

EPOLLOUT:

можно попробовать писать
write, скорее всего, не заблокируется

Но это не completion-модель.

Сравнение:

```
epoll:
    fd готов, теперь сам вызови read/write

io_uring:
    операция завершена, вот её результат
```

Поэтому после события `EPOLLIN` всё равно нужно вызвать `read`.

3. Почему fd всё равно делают неблокирующими

Может показаться:

Если `epoll` сказал, что fd готов, зачем `O_NONBLOCK`?

Причины:

1. Готовность могла измениться между `epoll_wait` и `read`.
2. Другой поток мог уже прочитать данные.
3. При edge-triggered режиме обязательно читать до `EAGAIN`.
4. Неблокирующий режим защищает event loop от случайного зависания.

Правило:

В event-driven сервере сокеты почти всегда переводят в `O_NONBLOCK`.

4. `select`

`select` — старый системный вызов для ожидания готовности fd.

Интерфейс примерно такой:

```
int select(
    int nfd,
    fd_set* readfds,
    fd_set* writefds,
    fd_set* exceptfds,
```

```
    struct timeval* timeout  
);
```

Использование:

```
fd_set readfds;  
FD_ZERO(&readfds);  
  
FD_SET(server_fd, &readfds);  
FD_SET(client_fd, &readfds);  
  
int max_fd = std::max(server_fd, client_fd);  
  
int ready = select(max_fd + 1, &readfds, nullptr, nullptr, nullptr);  
  
if (FD_ISSET(server_fd, &readfds)) {  
    // можно accept  
}  
  
if (FD_ISSET(client_fd, &readfds)) {  
    // можно read  
}
```

Что делает `select`:

пользователь передаёт набор fd
ядро проверяет готовность
поток спит, пока кто-то не готов
после возврата пользователь проверяет FD_ISSET

5. Проблемы `select`

1. Ограничение `FD_SETSIZE`

Обычно `fd_set` имеет ограниченный размер, часто около 1024 fd.

Это плохо для C10k.

```
10 000 соединений > 1024
```

2. Каждый раз нужно заново передавать набор fd

`select` портит `fd_set`: после возврата там остаются только готовые fd.

Поэтому перед каждым вызовом нужно заново собирать набор:

```
FD_ZERO(...)  
FD_SET(...)  
FD_SET(...)  
select(...)
```

3. $O(n)$ проверка

Даже после возврата нужно пройти по всем потенциальным fd и проверить:

```
FD_ISSET(fd, &readfds)
```

То есть всё равно есть линейный обход.

6. poll

`poll` — более современная альтернатива `select`.

Интерфейс:

```
struct pollfd {  
    int fd;  
    short events;  
    short revents;  
};  
  
int poll(struct pollfd* fds, nfds_t nfds, int timeout);
```

Пример:

```
std::vector<pollfd> fds;  
  
fds.push_back(pollfd{  
    .fd = server_fd,  
    .events = POLLIN,  
    .revents = 0  
});  
  
fds.push_back(pollfd{  
    .fd = client_fd,  
    .events = POLLIN,  
    .revents = 0  
});
```

```
});

int ready = poll(fds.data(), fds.size(), -1);

for (auto& p : fds) {
    if (p.revents & POLLIN) {
        // можно читать или accept
    }
}
```

Что улучшилось по сравнению с `select`:

нет жёсткого `FD_SETSIZE`
 удобнее работать с `vector<pollfd>`
 каждый fd имеет `events/revents`

Но главная проблема остаётся:

каждый вызов `poll` получает весь массив fd
 после возврата пользователь обходит весь массив
 сложность $O(n)$

7. Проблемы `poll`

`poll` лучше `select`, но для большого числа соединений всё ещё дорог.

Проблемы:

каждый раз передаём ядру весь массив fd
 ядро каждый раз просматривает весь массив
 пользователь после возврата тоже просматривает весь массив
 если активен 1 fd из 10 000 — всё равно $O(10\ 000)$

То есть `poll` хорошо снимает ограничение `FD_SETSIZE`, но не решает масштабирование полностью.

8. `epoll`: главная идея

`epoll` — Linux-механизм для эффективного ожидания множества fd.

Главное отличие:

В `select` / `poll` мы каждый раз передаём ядру весь набор fd.

В `epoll` мы один раз регистрируем интересующие fd в ядре, а потом просто ждём готовые события.

Схема:

```
epoll_create1:  
    создать epoll instance  
  
epoll_ctl:  
    добавить / удалить / изменить fd  
  
epoll_wait:  
    получить готовые события
```

То есть `epoll` хранит interest list внутри ядра.

9. Основные вызовы `epoll`

`epoll_create1`

```
int epfd = epoll_create1(EPOCH_CLOEXEC);
```

Создаёт epoll-instance и возвращает fd.

Можно думать так:

```
epfd – это fd специального объекта ядра,  
внутри которого хранится список отслеживаемых fd
```

`epoll_ctl`

```
int epoll_ctl(  
    int epfd,  
    int op,  
    int fd,  
    struct epoll_event* event  
);
```

Операции:

EPOLL_CTL_ADD – добавить fd
EPOLL_CTL_MOD – изменить интересующие события
EPOLL_CTL_DEL – удалить fd

Пример:

```
epoll_event ev{};
ev.events = EPOLLIN;
ev.data.fd = client_fd;

epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &ev);
```

epoll_wait

```
int epoll_wait(
    int epfd,
    struct epoll_event* events,
    int maxevents,
    int timeout
);
```

timeout:

-1 – ждать бесконечно
0 – не ждать, просто проверить
>0 – ждать столько миллисекунд

Пример:

```
epoll_event events[64];

int n = epoll_wait(epfd, events, 64, -1);

for (int i = 0; i < n; ++i) {
    int fd = events[i].data.fd;
    uint32_t ev = events[i].events;

    // обрабатываем готовый fd
}
```


10. События `epoll`

Основные события:

```
EPOLLIN   — можно читать
EPOLLOUT  — можно писать
EPOLLERR  — ошибка
EPOLLHUP  — hang up / соединение закрывается
EPOLLRDHUP — peer закрыл write-half соединения
EPOLLET   — edge-triggered режим
EPOLLONESHOT — после одного события fd отключается до повторного MOD
EPOLLEXCLUSIVE — будить одного epoll_waiter на shared fd
```

Чаще всего для TCP-сервера:

```
ev.events = EPOLLIN | EPOLLRDHUP;
```

Если есть данные на запись:

```
ev.events = EPOLLIN | EPOLLOUT | EPOLLRDHUP;
```

11. Level-triggered vs edge-triggered

Это очень важная разница.

Level-triggered

Это режим по умолчанию.

Идея:

```
пока fd остаётся готовым, epoll_wait будет снова и снова возвращать событие
```

Пример:

```
в сокете есть 100 байт
epoll_wait вернул EPOLLIN
мы прочитали только 50 байт
в сокете осталось 50 байт
следующий epoll_wait снова вернёт EPOLLIN
```

Плюс:

проще писать код
если не дочитал — событие придёт снова

Минус:

может быть больше повторных уведомлений

Edge-triggered

Включается флагом:

EPOLLET

Идея:

событие приходит только в момент перехода:
не было готово → стало готово

Пример:

в сокете появилось 100 байт
epoll_wait вернул EPOLLIN
мы прочитали только 50 байт
в сокете осталось 50 байт
нового перехода нет
следующий epoll_wait может не вернуть событие

Поэтому в edge-triggered режиме обязательно:

читать в цикле до EAGAIN
писать в цикле до EAGAIN
fd должен быть non-blocking

Шаблон:

```
while (true) {  
    ssize_t n = read(fd, buf, sizeof(buf));  
  
    if (n > 0) {  
        // обработать  
        continue;  
    }  
}
```

```

if (n == 0) {
    close_connection();
    break;
}

if (errno == EAGAIN || errno == EWOULDBLOCK) {
    break; // вычитали всё доступное
}

if (errno == EINTR) {
    continue;
}

close_connection();
break;
}

```

12. select vs poll vs epoll

Свойство	select	poll	epoll
Как передаются fd	<code>fd_set</code> каждый вызов	массив <code>pollfd</code> каждый вызов	регистрируются один раз через <code>epoll_ctl</code>
Ограничение fd	<code>FD_SETSIZE</code>	нет такого жёсткого	нет такого жёсткого
Проверка готовности	$O(n)$	$O(n)$	обычно $O(k)$, где k — число готовых событий
Возвращает готовые fd	нужно искать через <code>FD_ISSET</code>	нужно смотреть <code>revents</code> у всех	массив готовых событий
Удобство для C10k	плохо	лучше, но $O(n)$	хорошо
Портируемость	POSIX	POSIX	Linux-specific
Режим edge-triggered	нет	нет	да, <code>EPOLLET</code>

k — число реально готовых fd.

Главная формула:

```

select/poll:
    каждый раз передаём все fd и потом ищем готовые

epoll:

```

один раз регистрируем fd,
потом получаем только готовые события

13. Skeleton: однопоточный epoll-сервер

```
int server_fd = socket(AF_INET, SOCK_STREAM, 0);

set_nonblocking(server_fd);

bind(server_fd, ...);
listen(server_fd, SOMAXCONN);

int epfd = epoll_create1(EPOOL_CLOEXEC);

epoll_event ev{};
ev.events = EPOLLIN;
ev.data.fd = server_fd;

epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

std::unordered_map<int, Connection> conns;

epoll_event events[1024];

while (true) {
    int n = epoll_wait(epfd, events, 1024, -1);

    for (int i = 0; i < n; ++i) {
        int fd = events[i].data.fd;
        uint32_t event_mask = events[i].events;

        if (fd == server_fd) {
            // accept all ready connections
        } else {
            // handle client fd
        }
    }
}
```

14. Ассепт через epoll

Когда `server_fd` готов на чтение, значит в ассепт queue есть входящие соединения.

```
if (fd == server_fd) {
    while (true) {
```

```

int client_fd = accept(server_fd, nullptr, nullptr);

if (client_fd < 0) {
    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        break;
    }

    if (errno == EINTR) {
        continue;
    }

    // настоящая ошибка accept
    break;
}

set_nonblocking(client_fd);

epoll_event client_ev{};
client_ev.events = EPOLLIN | EPOLLRDHUP;
client_ev.data.fd = client_fd;

epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &client_ev);

conns.emplace(client_fd, Connection{client_fd});
}
}

```

Почему `accept` в цикле?

может быть несколько готовых соединений
особенно важно для edge-triggered
принимает до EAGAIN

15. Чтение client fd

```

void handle_read(Connection& c) {
    char buf[4096];

    while (true) {
        ssize_t n = read(c.fd, buf, sizeof(buf));

        if (n > 0) {
            c.in.append(buf, buf + n);

            // пример echo:
            c.out.append(buf, buf + n);
            continue;
        }
    }
}

```

```

    }

    if (n == 0) {
        c.closed = true;
        return;
    }

    if (errno == EAGAIN || errno == EWOULDBLOCK) {
        return;
    }

    if (errno == EINTR) {
        continue;
    }

    c.closed = true;
    return;
}
}

```

Даже в level-triggered режиме такой стиль нормален.

В edge-triggered — обязателен.

16. Запись client fd

Если у клиента есть данные на отправку:

```

void handle_write(Connection& c) {
    while (!c.out.empty()) {
        ssize_t n = write(c.fd, c.out.data(), c.out.size());

        if (n > 0) {
            c.out.erase(0, static_cast<size_t>(n));
            continue;
        }

        if (n < 0 && (errno == EAGAIN || errno == EWOULDBLOCK)) {
            return;
        }

        if (n < 0 && errno == EINTR) {
            continue;
        }

        c.closed = true;
        return;
    }
}

```

```
}  
}
```

Если `out` стал непустым после обработки чтения, нужно подписаться на `EPOLLOUT`.

Если `out` стал пустым, лучше отписаться от `EPOLLOUT`, иначе `fd` почти всегда будет считаться готовым к записи, и event loop будет просыпаться слишком часто.

```
void update_events(int epfd, Connection& c) {  
    epoll_event ev{};  
  
    ev.events = EPOLLIN | EPOLLRDHUP;  
  
    if (!c.out.empty()) {  
        ev.events |= EPOLLOUT;  
    }  
  
    ev.data.fd = c.fd;  
  
    epoll_ctl(epfd, EPOLL_CTL_MOD, c.fd, &ev);  
}
```

17. Заккрытие соединения

Если соединение закрыто:

```
void close_connection(int epfd, std::unordered_map<int, Connection>& conns,  
int fd) {  
    epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);  
    close(fd);  
    conns.erase(fd);  
}
```

Важно:

```
сначала удалить из epoll  
потом close  
удалить состояние соединения
```

На Linux `close` обычно удалит `fd` из `epoll` автоматически, но явно делать `EPOLL_CTL_DEL` полезно для ясности и контроля жизненного цикла.

18. Многопоточный сервер через epoll: варианты архитектуры

Есть несколько вариантов.

Вариант А. Один event loop + thread pool для CPU-задач

```
один поток:
  epoll_wait
  read/write
  парсинг минимальный
  тяжёлые задачи отправляет в thread pool

worker pool:
  делает CPU-heavy обработку
  возвращает результат в event loop
```

Плюсы:

```
простая модель владения fd
нет нескольких потоков, одновременно читающих один fd
меньше гонок
```

Минусы:

```
один event loop может стать bottleneck
надо аккуратно возвращать результаты из worker-ов
```

Это самый понятный учебный вариант.

Вариант В. Несколько event loop-ов, у каждого свой epoll

```
thread #1:
  epoll fd #1
  набор клиентов #1

thread #2:
  epoll fd #2
  набор клиентов #2

thread #3:
  epoll fd #3
  набор клиентов #3
```


Главный поток принимает соединения и распределяет `client_fd` между loop-ами.

Или каждый поток имеет свой listening socket через `SO_REUSEPORT`.

Плюсы:

масштабирование по ядрам
меньше contention
каждый fd принадлежит одному loop-у

Минусы:

сложнее балансировка
сложнее передача соединений между потоками

Вариант С. Несколько потоков делают `epoll_wait` на одном `epoll`

Это возможно, но сложнее.

Проблемы:

несколько потоков могут проснуться на одни и те же события
нужно не допустить параллельной обработки одного fd
thundering herd
нужны `EPOLLONESHOT` / `EPOLLEXCLUSIVE`

Для учебного ответа лучше описывать вариант А или В.

19. `EPOLLONESHOT`

Если несколько worker-ов делают `epoll_wait` на одном `epoll`-instance, можно использовать:

`EPOLLONESHOT`

Смысл:

после одного события fd автоматически отключается в `epoll`
пока его снова не включают через `EPOLL_CTL_MOD`

Это помогает гарантировать:

одно событие по fd обрабатывает один worker

Схема:

```
epoll_wait вернул fd
worker обрабатывает fd
пока он обрабатывает, epoll не отдаёт этот fd другим
после обработки worker делает EPOLL_CTL_MOD и rearm
```

Это похоже на “забрать fd в работу”.

20. EPOLLEXCLUSIVE

Если много потоков ждут на одном epoll/listening fd, может быть thundering herd.

EPOLLEXCLUSIVE говорит ядру:

```
разбуди одного или небольшое число ожидающих,
а не всех
```

Особенно важно для listening socket и многопоточного accept через epoll.

Не путать:

```
EPOLLONESHOT – контроль обработки конкретного fd одним worker-ом
EPOLLEXCLUSIVE – уменьшение thundering herd при пробуждении waiters
```

21. Реализация многопоточного сервера через epoll: учебная схема

Самая безопасная для объяснения схема:

```
main thread:
  создаёт server_fd
  делает server_fd non-blocking
  создаёт N event loop threads

каждый event loop thread:
  имеет свой epoll_fd
  обслуживает свой набор client_fd
```

приём соединений:

вариант 1 – main thread accett-ит и распределяет client_fd по loop-ам
вариант 2 – SO_REUSEPORT, у каждого loop-а свой server_fd

C SO_REUSEPORT:

у каждого потока свой listening socket
все bind на один port
ядро распределяет соединения между ассерт-очередями
каждый поток сам делает ассерт и добавляет клиентов в свой epoll

Это масштабируется лучше, потому что нет общего listening socket как bottleneck.

22. Схема event loop thread

```
void event_loop(int server_fd) {
    set_nonblocking(server_fd);

    int epfd = epoll_create1(EPOOL_CLOEXEC);

    epoll_event ev{};
    ev.events = EPOLLIN;
    ev.data.fd = server_fd;

    epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

    std::unordered_map<int, Connection> conns;
    epoll_event events[1024];

    while (true) {
        int n = epoll_wait(epfd, events, 1024, -1);

        for (int i = 0; i < n; ++i) {
            int fd = events[i].data.fd;
            uint32_t mask = events[i].events;

            if (fd == server_fd) {
                accept_ready_clients(epfd, server_fd, conns);
                continue;
            }

            auto it = conns.find(fd);

            if (it == conns.end()) {
                continue;
            }
        }
    }
}
```

```

        Connection& c = it->second;

        if (mask & (EPOLLERR | EPOLLHUP | EPOLLRDHUP)) {
            c.closed = true;
        }

        if (!c.closed && (mask & EPOLLIN)) {
            handle_read(c);
        }

        if (!c.closed && (mask & EPOLLOUT)) {
            handle_write(c);
        }

        if (c.closed) {
            close_connection(epfd, conns, fd);
        } else {
            update_events(epfd, c);
        }
    }
}

```

Это skeleton, который нужно уметь объяснить, а не обязательно писать идеально наизусть.

23. Почему `epoll` не делает всё за нас

`epoll` только говорит:

fd готов

Он не решает:

- как парсить протокол
- как хранить input/output buffers
- как делать backpressure
- как распределять CPU-heavy работу
- как не заблокировать event loop
- как безопасно закрывать fd

Даже с `epoll` остаются проблемы неблокирующего сервера:

- частичные read/write
- framing сообщений
- per-connection state

head-of-line blocking в event loop
backpressure на медленных клиентах

24. Типичные ошибки с epoll

Ошибка 1. Не сделать fd non-blocking

Особенно опасно в edge-triggered режиме.

Ошибка 2. В EPOLLET не читать до EAGAIN

Если прочитать не всё, нового события может не прийти.

Ошибка 3. Всегда слушать EPOLLOUT

Сокеты часто готовы к записи почти всегда. Если всегда подписаться на `EPOLLOUT`, event loop будет просыпаться постоянно.

Правило:

подписываемся на EPOLLOUT только когда output_buffer непустой
отписываемся, когда всё отправили

Ошибка 4. Делать долгую CPU-работу в event loop

Event loop должен быстро реагировать на I/O.

Тяжёлые задачи лучше отправлять в thread pool.

Ошибка 5. Несколько потоков обрабатывают один fd

Нужно владение fd одним event loop-ом или механизмы вроде `EPOLLONESHOT`.

25. Что сказать на экзамене: короткая версия

Можно отвечать так:

После неблокирующих сокетов busy loop polling всё ещё плох: программа сама обходит все fd, получает много `EAGAIN`, жжёт CPU и делает $O(n)$ -опрос. Поэтому используют I/O multiplexing: `select`, `poll`, `epoll`.

`select` принимает наборы fd через `fd_set`, ждёт готовности и после возврата надо проверять `FD_ISSET`. У него есть ограничение `FD_SETSIZE`, и каждый вызов требует заново передавать и просматривать набор fd.

`poll` использует массив `pollfd`, снимает жёсткое ограничение `FD_SETSIZE`, но всё равно каждый раз передаёт весь массив fd, ядро и пользовательский код проходят по нему линейно. Поэтому при 10 000 соединениях и одном активном fd это всё ещё $O(n)$.

`epoll` работает иначе: мы создаём epoll-instance через `epoll_create1`, добавляем fd через `epoll_ctl`, а потом `epoll_wait` возвращает массив уже готовых событий. Поэтому не нужно каждый раз передавать все fd, и работа ближе к $O(k)$, где k — число готовых событий. `epoll` — Linux-specific и хорошо подходит для C10k.

В `epoll` есть level-triggered и edge-triggered режимы. Level-triggered повторяет событие, пока fd готов. Edge-triggered сообщает только о переходе “не готов → готов”, поэтому fd обязательно должен быть non-blocking, и читать/писать надо до `EAGAIN`.

Многопоточный сервер через `epoll` можно строить так: несколько event loop-потоков, у каждого свой epoll-instance и свой набор клиентских fd. Приём соединений можно распределять через главный ассерт-поток или через `SO_REUSEPORT`, где у каждого worker-а свой listening socket на одном порту. Если несколько потоков ждут на одном epoll/listening fd, нужны механизмы вроде `EPOLLONESHOT` или `EPOLLEXCLUSIVE`, чтобы избежать параллельной обработки одного fd и thundering herd.

26. Мини-код для ответа на доске

```
int epfd = epoll_create1(EPOOL_CLOEXEC);

epoll_event ev{};
ev.events = EPOLLIN;
ev.data.fd = server_fd;

epoll_ctl(epfd, EPOLL_CTL_ADD, server_fd, &ev);

epoll_event events[1024];

while (true) {
    int n = epoll_wait(epfd, events, 1024, -1);

    for (int i = 0; i < n; ++i) {
        int fd = events[i].data.fd;
        uint32_t mask = events[i].events;

        if (fd == server_fd) {
```

```

while (true) {
    int client_fd = accept(server_fd, nullptr, nullptr);

    if (client_fd < 0) {
        if (errno == EAGAIN || errno == EWOULDBLOCK) {
            break;
        }

        if (errno == EINTR) {
            continue;
        }

        break;
    }

    set_nonblocking(client_fd);

    epoll_event cev{};
    cev.events = EPOLLIN | EPOLLRDHUP;
    cev.data.fd = client_fd;

    epoll_ctl(epfd, EPOLL_CTL_ADD, client_fd, &cev);
}
} else {
    if (mask & EPOLLIN) {
        // read until EAGAIN
    }

    if (mask & EPOLLOUT) {
        // write until EAGAIN
    }

    if (mask & (EPOLLERR | EPOLLHUP | EPOLLRDHUP)) {
        epoll_ctl(epfd, EPOLL_CTL_DEL, fd, nullptr);
        close(fd);
    }
}
}
}
}

```

Что объяснить:

epoll_create1 создаёт epoll-instance
 epoll_ctl регистрирует fd и интересующие события
 epoll_wait возвращает готовые события
 server_fd готов на EPOLLIN → можно accept
 client_fd готов на EPOLLIN → можно read
 client_fd готов на EPOLLOUT → можно write
 fd должны быть non-blocking

27. Мини-проверка

1. Зачем нужны `select`, `poll`, `epoll` после busy loop polling?
2. Что значит “fd готов к чтению”?
3. Почему в epoll-сервере fd всё равно делают non-blocking?
4. Какие главные проблемы `select`?
5. Чем `poll` лучше `select`?
6. Почему `poll` всё равно плохо масштабируется?
7. В чём главная идея `epoll`?
8. Какие три основных вызова есть у `epoll`?
9. Чем level-triggered отличается от edge-triggered?
10. Почему в edge-triggered режиме надо читать до `EAGAIN`?
11. Почему не стоит всегда подписываться на `EPOLLOUT`?
12. Как можно организовать многопоточный сервер через `epoll`?